

UNIVERSIDADE
FEDERAL DO CEARÁ

Test Smells em projetos OpenSource

Disciplina: Qualidade de Software
Equipe: Pedro Rivaldo (individual) - 511881
Projeto: https://github.com/HKUDS/LightRAG
Fork: https://github.com/pedrorivald/LightRAG

Exception Handling

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **tests/test_create_prefixed_exception.py:26`, `test_jsondecodeerror_does_not_raise_and_preserves_message`**
Minhas principais dificuldades na remoção do code smell são: nenhuma relevante. Notei que, no código original, se `json.loads` deixasse de lançar a exceção, o teste quebraria com um `NameError` (variável `result` não definida), uma mensagem confusa.
Usei o CATSPY para: definição e exemplo de refatoração (`assertRaises` → equivalente no pytest: `pytest.raises`).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei o `try/except` por `with pytest.raises(json.JSONDecodeError) as exc_info:` e passei a usar `exc_info.value`.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **`tests/kg/json_impl/test_atomic_write_write_json.py:55`, `test_write_json_concurrent_writers_land_intact`**
Minhas principais dificuldades na remoção do code smell são: o exemplo do CATSPY é de uma exceção esperada. Aqui a exceção é inesperada e acontece dentro de uma thread, onde o pytest não a vê. Foi preciso achar outro mecanismo para propagar o erro, e também garantir que a `Barrier(5)` continuasse funcionando dentro de um pool.
Usei o CATSPY para: definição (o teste fazia um tratamento de exceção manual que o framework/biblioteca já resolve).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei as threads manuais por `ThreadPoolExecutor(max_workers=5)` + `list(pool.map(...))`, que relança a exceção do worker com o traceback original. Também removeu 2 Conditional

Test Logic (laços de `start`/`join`).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **`tests/kg/networkx_impl/test_networkx_atomic_write.py:50`, `test_write_nx_graph_concurrent_writers_no_race`**
Minhas principais dificuldades na remoção do code smell são: nenhuma nova; reaproveitei a solução anterior.
Usei o CATSPY para: definição.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: ThreadPoolExecutor + `pool.map`. Também removeu 2 Conditional Test Logic.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **`tests/pipeline/test_pipeline_release_closure.py:1066`, `test_resume_skips_purge_when_chunks_list_empty`**
Minhas principais dificuldades na remoção do code smell são: era preciso saber se o pipeline lança exceção nesse cenário. Investiguei e vi que ele registra a falha de cada documento no `doc_status` em vez de lançar.
Usei o CATSPY para: definição e efeito negativo ("pode ofuscar a intenção do teste e reduz a confiabilidade").
Eu estou usando os seguintes métodos de refatoração para remover o code smell: removi o `try/except/pass` e deixei um comentário explicando por que a chamada não precisa de proteção. Também removeu 1 Non-Functional Statement (o `pass`).

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **`tests/llm/test_tokenizer_contract.py:142`, `test_builtin_tokenizer_survives_concurrent_encode_and_decode`**
Minhas principais dificuldades na remoção do code smell são: separar as duas coisas, erro e resultado errado, sem criar novos `for`/`if` dentro do teste, que gerariam Conditional

Test Logic.
Usei o CATSPY para: definição.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: cada worker passou a *retornar* seus pares (tokens, texto decodificado), feitos com list comprehension. `pool.map` relança as exceções, e uma única asserção compara tudo com o resultado de referência, calculado sem threads. Também removeu 5 Conditional Test Logic.

Eu estou atualmente trabalhando na refatoração do seguinte code smell: **`tests/pipeline/test_tolerant_load_json_dict.py:60`, `test_greedy_regex_would_fail_same_input`**
Minhas principais dificuldades na remoção do code smell são: este foi o mais difícil. A sugestão do CATSPY (`pytest.raises`) não se aplica: rodando, vi que o `json_repair` não lança exceção nessa entrada (ele retorna uma lista). O `try` existe para tolerar versões da biblioteca que lancem.
Usei o CATSPY para: definição. A refatoração sugerida não servia, e precisei adaptar.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: A lógica do parser antigo foi para a função auxiliar `_legacy_greedy_load(raw)`, e o teste ficou com duas asserções que expressam a afirmação: o parser antigo perde o objeto, o novo recupera. Uma pequena mudança de semântica: se a regex não encontrar nada, a função auxiliar retorna `{}`, que é o comportamento real do parser antigo.

Conditional Test Logic

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Conditional Test Logic de modo geral **`test_bool_index_is_rejected_even_though_bool_is_an_int_subclass`** **`test_sanitize_filename_rejects_separator_and_control_characters`** **`test_sanitize_filename_rejects_colon`** **`test_sanitize_filename_allows_other_windows_reserved_characters`** **`test_empty_addon_params_falls_back_to_env_default`** **`test_env_bool_truthy_falsy`** **`test_env_bool_truthy`** **`test_env_bool_falsy`** **`test_non_string_path_value_is_refused_not_raised`** **`test_reservation_owner_kind_reads_a_full_owner_record`**

Minhas principais dificuldades na remoção do code smell são: pouca. Precisei cuidar para os testes dentro de classe (`self`) e com fixtures (`tmp_path`, `monkeypatch`) receberem o parâmetro na ordem certa, e deixar o ruff reformatar as assinaturas.
Usei o CATSPY para: definição e refatoração. O catálogo sugere separar os casos em testes distintos; no pytest, a forma idiomática disso é `@pytest.mark.parametrize`, que gera um caso por valor.
Eu estou usando os seguintes métodos de refatoração para remover o code smell: Parametrize Test (separar os casos do laço em testes independentes). Efeito nas métricas: o número de testes coletados pelo pytest aumenta, porque cada valor vira um caso. É esperado; explique no relatório.

Unknown Test

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Unknown Test de modo geral
Minhas principais dificuldades na remoção do code smell são: quase todos eram testes do tipo "não deve lançar exceção". O desafio foi achar uma asserção que não fosse trivial (como `assert f() is None`) e que acrescentasse garantia de verdade. Para isso li a implementação de cada função testada.
Usei o CATSPY para: definição ("o teste será aprovado se nenhuma exceção for lançada") e refatoração (adicionar asserções explícitas sobre o resultado esperado).
Eu estou usando os seguintes métodos de refatoração para remover o code smell: **`tests/parser/external/test_common.py:76`, `test_clear_dir_contents_noop_when_missing`** adicionei `assert not missing.exists()`. "No-op" também significa não criar o diretório. **`tests/kg/test_bounded_pipeline_history.py:174`, `test_a_missing_history_is_skipped_not_raised`** guardo um `copy.deepcopy(status)` antes da chamada e verifico `status == before`. "Skipped" significa que nada foi criado nem escrito. **`tests/kg/test_bounded_pipeline_history.py:178`, `test_a_failing_extend_is_swallowed`** passei a guardar a referência ao proxy e verifico `history == []`, ou seja, a falha é engolida sem nenhuma escrita alternativa.

`tests/kg/test_pipeline_status_logger.py:229`, `test_never_raises_even_when_diagnostic_logging_itself_fails`
três asserções. A falha injetada foi consumida (`fail_next_gets == 0`), o `latest_message` foi escrito e o histórico não recebeu nada.

`tests/parser/docx/test_zip_budget.py:512`, `test_validate_source_does_not_open_the_archive`
o `ZipFile` falso agora **registra** cada chamada numa lista, além de lançar exceção, e o teste verifica `opened == []`.

`tests/parser/docx/test_zip_budget.py:372`, `test_ordinary_docx_passes_untouched`
comparo os bytes do arquivo antes e depois da validação. O nome do teste promete "untouched", e nada verificava isso.

Non-Functional Statement

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
`tests/kg/test_scheduling_backend_fixture.py:67`, `test_isolation_leaves_an_unset_variable_unset`

Minhas principais dificuldades na remoção do code smell são:

Usei o CATSPY para: refatoração ("adicionar a respectiva asserção").

Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei o `pass` pela asserção que o teste vizinho faz no mesmo lugar: a variável fica ausente durante o isolamento, além de depois.

Eu estou atualmente trabalhando na refatoração do seguinte code smell:
`tests/llm/gemini_impl/test_gemini_llm.py:206`, `FakeSchemaModel`

`tests/api/test_custom_chunker_startup.py:51`, `Constructed`

`tests/pipeline/test_pipeline_release_closure.py:989`, `_ResumePurged`



UNIVERSIDADE
FEDERAL DO CEARÁ

Minhas principais dificuldades na remoção do code smell são: o CATSPY trata o `pass` como placeholder de teste não implementado. Aqui o `pass` está no corpo de classes dublê, onde é sintaxe necessária, não um teste vazio. Portanto, é um caso de fronteira: a ocorrência é real para o TEMPY, mas o impacto é baixo. A melhoria possível foi deixar explícita a intenção.
Usei o CATSPY para: definição ("pode confundir outros desenvolvedores").
Eu estou usando os seguintes métodos de refatoração para remover o code smell: troquei o `pass` por uma docstring de uma linha explicando o papel do dublê. Por exemplo: "Raised after the real purge ran, ending the pipeline there."